Ouroboros v7.1 — A Student's Guide

What we are building, every concept explained from scratch, how the pieces fit, what we achieved, and the complete roadmap of future phases.

> **Who this is for:** a computer-science student who knows data structures, basic OS concepts (virtual memory, processes) and a little Python — but maybe not machine learning, GPUs or formal verification. Every specialised term is explained when it first appears, and again in the glossary (§9).

1 · The story in one page

The problem. Today's AI coding assistants write code one token at a time, left to right. That is fine for autocomplete, but useless for *refactoring* — restructuring existing programs (renaming, extracting methods, splitting classes). Editing anything forces the model to regenerate everything after the edit point, and its left-to-right bias means it never plans globally.

The idea. Borrow **diffusion models** (the technique behind image generators): start with a corrupted/blanked program and iteratively denoise it toward a corrected version — editing *anywhere, in any order*. Add two magic control tokens: `[EXPAND]` ("make room here") and `[DELETE]` ("remove this").

The wall. Real programs grow and shrink while they are being generated. On a GPU, changing a tensor's size mid-computation is catastrophic: it forces reallocation of all memory, destroys caches, and breaks CUDA Graphs (pre-recorded GPU command sequences that make inference fast).

Our answer — Ouroboros v7.1: seven cooperating modules that give diffusion models room to grow on real hardware, prove their edits correct with mathematical tooling, survive kernel crashes, and learn where to expand — all governed by four inviolable architectural laws enforced by AI agents during development.

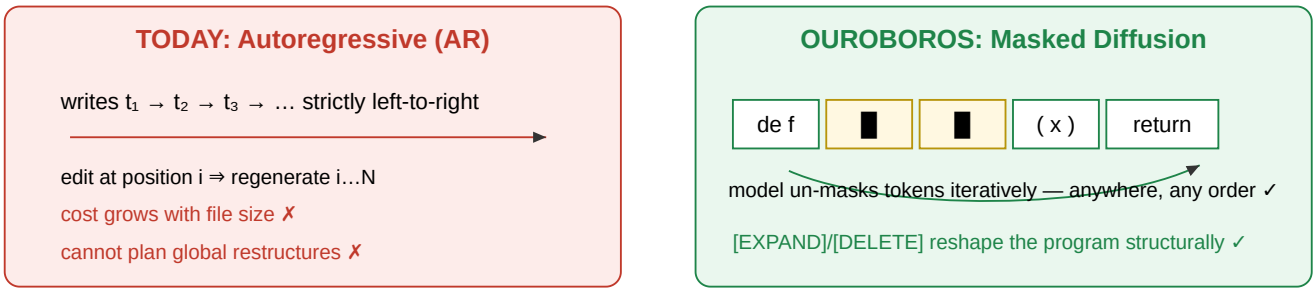

Figure 1 — Why we switched paradigms: parallel any-order refinement vs sequential regeneration.

2 · Background concepts (everything you need)

2.1 Tokens & tokenisation

Token: a chunk of text the model reads/writes — a word piece, symbol or keyword. `"x = a + 1"` might become tokens `[x, =, a, +, 1]`. Models never see characters; they see integer IDs looked up in a vocabulary table (`Vocab` maps `"x"` → 7 etc.). Our tokenizer assigns consecutive IDs starting at 1 (0 is reserved as padding).

2.2 Abstract Syntax Trees (AST)

AST: the parser's tree view of a program. `x = a + 1` becomes an Assign node whose children are a Name (`x`) and a BinaryOp (`a + 1`). Python's built-in `ast` module gives us this for free. The tree encodes *structure*: which expression lives inside which function, who is the parent of whom.

We use the AST twice: (a) to attribute each token to its smallest containing node, and (b) to compute the **shortest-path-distance matrix φ(i,j)** between all token pairs by BFS over the undirected parent↔child edges. Two tokens that sit in the same tiny expression have small φ even if they are far apart in the raw text — exactly the "these belong together" signal attention needs.

2.3 Attention, softmax and positional encodings

Self-attention: every position builds a *query* q and a *key* k from its embedding; the score between positions i and j is $\langle q_i, k_j \rangle / \sqrt{d}$. A **softmax** turns scores into weights summing to 1, and the output is the weighted average of *value* vectors v_j. So attention literally asks "who should I look at?".

RoPE (Rotary Position Embedding): a clever way to inject *position* into attention: rotate query/key vector pairs by an angle proportional to position. The dot product then depends only on the *relative offset* m−n between tokens (Eq. 3 of the paper). We use the standard 1-D version on lexical order — and our law guard makes 2-D experiments structurally impossible (§4, M3).

2.4 Diffusion models for code

Masked discrete diffusion: take a correct program, progressively replace tokens with `[MASK]`; train a network to predict originals given the partially-masked view. At inference, start fully masked and un-mask step by step — **in any order**, because every position is always visible. Ouroboros adds two structural actions: `[EXPAND]` (insert k fresh masks here — the program needs more room) and `[DELETE]` (this span is redundant).

2.5 GPUs, VRAM and CUDA Graphs

*Think of GPU memory (VRAM) as a huge warehouse and CUDA kernels as forklift instructions. A **CUDA Graph** is a pre-recorded forklift choreography replayed with near-zero CPU overhead — but the choreography assumes shelves never move. Resizing a tensor mid-run = rebuilding half the warehouse while forklifts run: everything stalls and cached routes die.*

- **Why reshapes hurt:** PyTorch must allocate brand-new contiguous storage and copy — O(N), plus the old layout's cache locality is lost.

- **Why static shapes matter:** CUDA Graph capture records exact pointer addresses; dynamic sizes invalidate the recording.
- **Triton:** a language for writing GPU kernels where you program tiles of work; `make_block_ptr` lets a kernel treat scattered memory blocks like tidy tiles (paper §4.3).
- **HBM vs SRAM:** big-but-slow GPU memory vs tiny-but-fast on-chip memory. Fused kernels load once into SRAM and do RoPE + bias + masking *inside* before writing back.

2.6 Virtual memory & paging → block tables

Paging (from OS class): processes see contiguous virtual addresses; the OS maps pages to scattered physical frames via a page table. vLLM applied this to KV-caches. Ouroboros applies it to the *sequence itself*: 64-token physical blocks, chained through an array-backed linked list called the **block table**. Growth = allocate one block, update one pointer. No resizes, ever.

2.7 e-graphs & Rice's theorem

e-graph: a data structure that compactly stores *many equivalent versions* of a program at once, grouping equal expressions into e-classes. **Equality saturation:** apply rewrite rules ($a+b \Leftrightarrow b+a$, De Morgan…) exhaustively until equivalent forms meet in one e-class — proving equality without choosing an order. **Rice's theorem:** any non-trivial semantic property of programs is undecidable — so saturation may never terminate. Answer: hard resource caps + asynchronous execution + honest "yellow" results.

2.8 RL, GRPO and antithetic variates

REINFORCE: sample actions from the policy, weight each sampled action's log-probability by the reward, ascend. **GRPO:** compare rewards within a group to form advantages (no value network needed). **Antithetic variates:** evaluate pairs of complementary configurations (here: perfectly complementary masks $m^1 \cup m^2 = 1$, $m^1 \cap m^2 = \varnothing$) so randomness cancels — variance drops. Our advantage is $r^1 - r^2$ per couple.

2.9 SSA form, PyO3 & maturin

SSA (Static Single Assignment): every variable is assigned exactly once (x_1, x_2, …). Makes data-flow explicit and analysis easy. **PyO3:** Rust ↔ Python bindings; **maturin** builds a Rust crate into a pip-installable wheel. Our verifier core is Rust (fast, safe); Python calls it through this bridge.

3 · The seven modules — problem → idea → proof

Each module below follows the same pattern: the obstacle, the mechanism, and the test that pins it. Test counts come from the verified suites (§5).

M1 · Tokenizer, Phantom Padding & the Algorithm-1 loop (ouroboros-core — 143 tests)

Obstacle: sequences must grow/shrink while the physical buffer stays exactly L_MAX=1024 tokens.

Mechanism: front-pack tokens, track a *logical length*, and implement growth/shrinkage as in-place operations: `insert_masks(pos, k)` converts tail capacity into fresh masks at the splice point; `logical_delete(pos)` pops and refills with `[IGNORE]`; `derive_mask()` yields the attention mask. The Algorithm-1 loop then runs: policy inspects the buffer, picks KEEP/EXPAND/DELETE per masked slot, and the shape never changes.

We also completed the AST-aware tokenizer itself (`tokenize_with_nodes`, shared `Vocab`) and the φ-matrix helper — 18 tests.

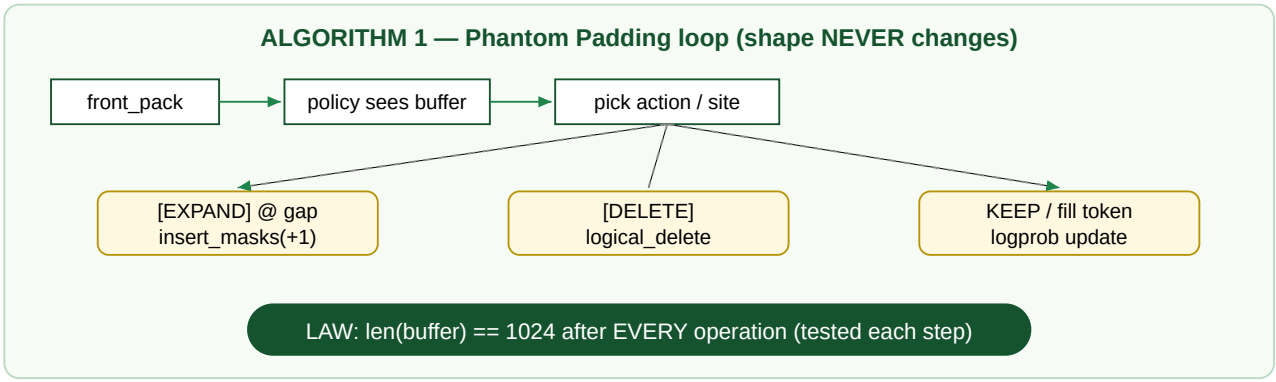

Figure 2 — The loop that never resizes: growth is bookkeeping, not memory movement.

M1-RL · Coupled-GRPO training (143 → 163 tests)

Obstacle: RL needs reward signal; real verification is undecidable and slow.

Mechanism: antithetic mask pairs (complementary by construction), rewards from the Fuzzy Proxy formula, advantages $r^1 - r^2$, and a loss that touches *only* the gap position's log-probability. Our Wave-8 learning proof: held-out EXPAND-at-gap accuracy went from 0.107 → 0.209 (2.0×, independently reproduced); later, swapping in the attention placement head pushed absolute accuracy to 0.40 (≈5× fresh init) — breaking the scaffold ceiling documented in DEFERRED.md §6.

M3 · Attention math (core — inside the 163-test suite)

Obstacle: injecting AST topology without breaking hardware-friendly attention.

Mechanism: scores become Softmax(QK/√d + **b_φ(i,j)**) with b_φ = log1p(shortest-path distance) — additive, differentiable, no message-passing GNN. A structural guard raises if anyone attempts a 2-D position grid (the banned math-rope violation). Tests include the relative-offset property (score depends only on m−n), exact additivity against manual computation, and scope-mask integration.

M4 · Block-sparse lexical scoping (core + triton — aligned bit-for-bit)

Obstacle: bidirectional diffusion wants full attention; IDE latency forbids it.

Mechanism: Table-1 hierarchy — LOCAL sees GLOBAL and itself and its own scope; GLOBAL ignores LOCAL (freezing the global KV-cache!); sibling scopes don't bleed. Implemented identically in both repos; 21+ tests including a hand-written 8×8 literal matrix and asymmetry proofs.

M2 · Block-level memory & chains (triton — 121 tests incl. demo)

Obstacle: growth-by-resize destroys everything (§2.5).

Mechanism: slot-identity BlockTable (id == storage index forever — kills a real desync bug found in the skeleton), all-or-nothing chain allocation, `expand_chain` as the [EXPAND] primitive, double-free rejection. Differential parity against a C++ twin is scaffolded (skipped until Python.h exists on the host).

M2-GPU · The Triton kernel (GPU-proven on T4)

Obstacle: attention over SCATTERED blocks needs pointer-chasing plus three fused operations, and Triton's compiler rejects plain `continue`.

Mechanism: per-block program loads the physical id from the page table, builds `make_block_ptr` views into the scattered cache, rotates Q/K with RoPE tables, adds bias, applies −1e30 masking (finite negatives instead of −inf — avoids NaN poisoning when a leading tile is fully scoped-out), online softmax, writes through an output block ptr. Verified against the pure-Python golden reference on a real T4: **120 passed / 6 skipped / 0 failed**. The permutation study proved the reference ↔ kernel channel mapping is the out-shuffle (involution only at $d \in \{2,4\}$; explicit inverse required at d=8 — 6e-8 agreement).

M5 · SSA → egg → telemetry (dfg — 86/86)

Obstacle: prove refactorings semantically equivalent without hanging or lying.

Mechanism: lower Python subsets to SSA IR → render to egg expressions → saturate under LAW-pinned limits (5000 iterations / 10 s / 1 M nodes / backoff 5000·3) → verdicts are honest: Equivalent (green) or Unproven (yellow) — yellow is a designed outcome feeding the SRE ratio. Runs async; exposed to Python via PyO3: `ouroboros_dfg.prove_refactor(src_a, src_b)`. Tree-sitter frontend lowers real assignments/arithmetic into the SSA IR.

M6 · Supervisor–Worker HA (triton — demo with 6 invariant proofs)

Obstacle: custom kernels segfault; a crashed node cannot serve.

Mechanism: the Supervisor solely owns the block table + KV-cache (workers get a single mediated door, `table_op`); workers are stateless (purity test-enforced), so restart = fresh object; crashes degrade exactly one request into the Safe Queue (FIFO, drained exactly once); a 12-request demo session proves zero block leaks and blast radius = 1.

4 · The governance layer (M7) — how agents build this safely

Everything above was written by AI specialists under five enforced properties:

1. **Law validation before implementation** — intents checked against laws.json by the reasoning-model validator (ALLOW/BLOCK recorded).
2. **Test-first discipline** — red phases confirmed before green across waves.
3. **Independent orchestrator verification** — agent claims accepted only after reproduction (several were rejected and repaired — see Campaign Report §4).
4. **Deferral honesty** — impossible-here items documented with unblock steps, never faked.
5. **Institutional memory** — WAVELOG/CAMPAIGN ledgers survive session turnover.

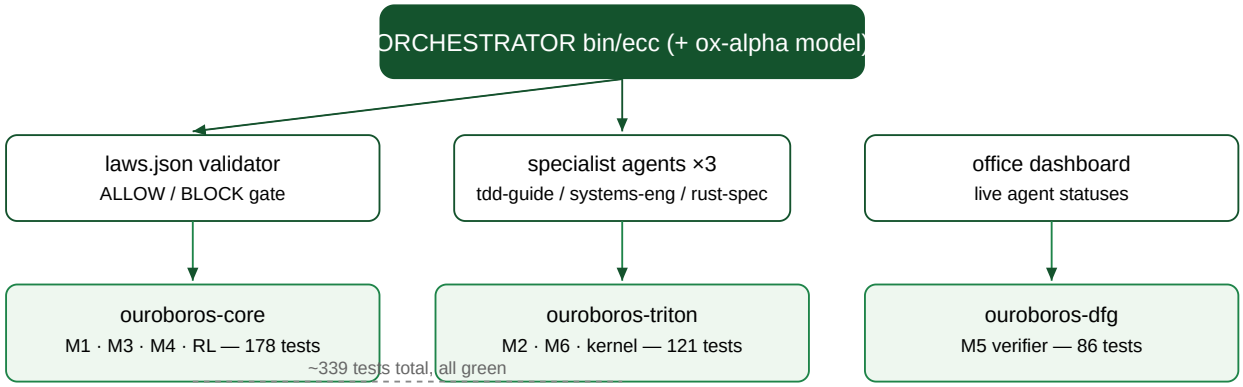

Figure 3 — Governance layer: laws gate the swarm; the swarm fills the repos; the office watches.

5 · What we achieved (verified numbers)

Claim	Evidence
Suites green	core 178 · triton 121+10skip · dfg 86 — all orchestrator-re-run
Triton kernel proven on GPU	T4: 120 passed/6 skipped/0 failed; kernel≡golden after 3 hardware-caught fixes
RoPE channel semantics pinned	d=8 study: permuted 6e-8 PASS vs un-permuted 8.3e-2 FAIL
Policy learns	held-out lift ×2.0 (orchestrator) / ×2.54–3.38 (5 seeds); attention head +0.156 abs on unseen seeds
Source-level proofs from Python	`prove_refactor("(r=(a+b)+c)", "(r=a+(b+c))") → equivalent`
HA invariants	demo session: served-exactly-once, blast-radius-1, zero leaks
Laws enforced	ALLOW×N recorded; BLOCK would stop specialists pre-write

6 · Teaching moments — real bugs we caught (and what they teach)

Bug	The lesson
BlockTable ids desynced from storage slots after free→realloc	Data-structure invariants must be TESTED, not assumed; identity mappings beat clever indirection
RoPE broadcast [T,1,hf] vs [H,T,hf]	Tensor broadcasting bugs are silent — property tests catch them
Triton rejects `continue`	DSL compilers have grammar limits — know them before designing loops
−inf − (−inf) = NaN poisoning	FlashAttention-style accumulators need FINITE negative floors (−1e30), not −inf
Cross-graph param naming collision	Identity must come from MEANING (declaration ordinal), not allocation accident
AirTel DNS poisoning of tunnel services	Infrastructure hostility is real; CDN mirrors + DoH IP pinning are the tools

7 · Future phases — the detailed plan

PHASE 1-C · Finish the scaffold (now → short)

Goal: close remaining local gaps.
Tasks:

- Placement-head scaling study on Colab T4: sweep d_model ∈ {16,32,64} × epochs {5,10,20}; publish the accuracy-vs-config table.
- Activate C++ parity locally: `sudo apt install python3.12-dev` (user action) → build.sh → differential suite runs.
- Permutation-aware comparison already lands for d≤32; extend golden oracle to multi-token-per-block (removing the documented simplification).
- Wire `ecc verify-manifest` pairs automatically from agent landings (currently semi-manual JSON).

Exit criteria: zero code skips except GPU legs; parity suite live; scaling table committed.

PHASE 2-A · Make the toy policy REAL (Colab T4)

Goal: replace PhantomPolicy (mean-pool stub) with a small transformer diffusion model that actually denoises code buffers.
Tasks (ordered):

- Mini-diffusion backbone: 4-layer transformer, d_model=128, rotary positions, vocab = our tokenizer Vocab (~few hundred entries at toy scale).
- Training objective: masked-token prediction on stage-1 curriculum instances (SFT warmup, paper curriculum phase 1).
- Coupled-GRPO fine-tune: antithetic pairs from grpo.py; rewards = fuzzy proxies computed by ACTUALLY parsing/type-checking outputs (ast.parse + a type stub pass).
- Success metric: on held-out corruption instances, EXPAND-at-gap placement ≥ 2× the SFT-only checkpoint, and generated completions parse ≥ 95%.
- Deliverables: checkpoint_w9.pt, training curves, ablation (no antithetic pairing vs paired — validating the variance-reduction claim empirically).

Compute: T4 16 GB suffices (model ≤ 25 M params); est. 4–8 GPU-hours.
Risks: reward sparsity → mitigate with curriculum staging already built; overfitting tiny data → hold-out gates exist.

PHASE 2-B · Kernel meets the real model (T4 → local RTX 3050)

Goal: run the trained model's attention THROUGH the block-sparse Triton path end-to-end.
Tasks:

- Install CUDA-torch + triton locally (parallel-resume downloader pattern proven; ~2.5 GB).
- Convert PhantomLoop's logical operations to block-table operations (bridge exists conceptually; implement + differential test vs Python reference).
- Capture one 64-token AOT graph; replay across chains; validate vs eager mode bitwise-bounded.
- Latency budget measurement against the paper's sub-200 ms target.

Exit criteria: identical outputs (tolerance-bounded) between dense reference and block-sparse path on random tables at d_head=32; latency measured and logged.

PHASE 3 · Serving & IDE prototype

Goal: the Module-6 story made real for a user.
Tasks:

- Package supervisor+workers as a service (FastAPI wrapper around serving.py semantics; workers as separate processes).
- Kill-and-respawn chaos testing (automated: kill -9 workers mid-stream; assert Safe Queue absorbs all).
- Minimal VS Code extension: select code → send to service → receive refactor + green/yellow badge from the dfg verifier.
- Office dashboard shows live requests (bridge already speaks JSON).

Exit criteria: a human edits code in the editor, gets a refactor with a mathematically-verified badge or an honest warning.

PHASE 4 · Research artefacts

- Benchmark suite: refactor-equivalence corpus (before/after pairs graded by verify_ssa) — the green/yellow ratio becomes measurable.
- Ablations: antithetic vs independent pairs; coupled vs uncoupled rewards; BackoffScheduler limit sweeps (feeds the SRE tuning story).
- Paper results section assembled from WAVELOG numbers + new benchmarks.

STRETCH · Hardware-bound items (await resources)

- GB10 Grace Blackwell ATS/TLB warmup reproduction (logic ships today; needs the silicon).
- C++ BlockTable parity execution (needs `sudo apt install python3.12-dev` on some box).
- Full-scale GRPO (>4 GB VRAM policy training).

8 · Glossary

Term	Meaning
Token / vocab	A chunk of text and its integer ID in the model's dictionary
AST	Abstract Syntax Tree — the parser's structural view of code
Masked diffusion	Generative modelling by iteratively un-masking a corrupted sequence
[EXPAND]/[DELETE]	Control tokens letting the model grow/shrink the sequence structurally
Phantom Padding	Fixed-size buffer + logical mask: growth without resizing memory
Block table	Array-backed linked list mapping logical sequence chunks → physical 64-token blocks
CUDA Graph	Pre-recorded GPU kernel sequence replayed with minimal CPU overhead
RoPE	Rotary positional embedding: rotation making scores depend on relative distance
Additive AST bias b_φ(i,j)	log-compressed shortest-path distances injected additively into attention scores
Block-sparse scoping	Mask letting LOCAL regions attend globally+within-scope only; GLOBAL stays frozen
e-graph / egg	Structure storing many equivalent expressions; equality saturation applies rewrite rules exhaustively
Rice's theorem	Any non-trivial semantic property of programs is undecidable — hence caps + async verification
SSA	Static Single Assignment: every value assigned exactly once
Coupled-GRPO	RL scheme using complementary mask pairs so two views share one sequence's information
Fuzzy Proxy reward	0.1·Parses + 0.3·TypeChecks + 0.6·PassesTests — fast heuristic stand-in for correctness
Safe Queue	Degradation path: crashed-worker requests rerun on the dense Phase-1 backend
Green/Yellow ratio	Fraction of refactors formally proven vs merely test-passed; SRE-tuned
PyO3 / maturin	Rust ↔ Python bindings / wheel builder exposing the verifier to Python
BackoffScheduler	egg scheduler banning over-firing rewrite rules temporarily (match 5000, ban 3)

Companion documents: ***Campaign Report*** *(what we did, defect log, GPU milestone) and* ***Architecture & Novelty Analysis*** *(positioning vs prior art). Ledgers live in* `.ecc/reports/`.